DESIGN & ANALYSIS OF ALGORITHMS
PROJECT REPORT SUBMISSION FORM

EVIDENCE SHEET

Assignment (1-6): 3	**Course / section:** DAA / AIML Panthers	**Submitted on:** 14 Oct 2026
Project / framework title: DAA Algorithm Vulnerability and Stress-Testing Framework (Assignment 3)		
Team members and roll numbers: Nabiha Fatima - 25WU0102172; Kasani Hansith Raj - 25WU0102117; Maydhansh Nanda - 25WU0102155; Mithi Jaiswal - 25WU0102158; Mayank Panigrahi - 25WU0102154		
Code / repository URL: HansithKasani/DAA-PBL-: Algorithm Stress testing and vulnerability	**Report / demo link:** DAA Algorithm Vulnerability & Stress-Testing Laboratory	

PURPOSE Demonstrate how you model a real problem, justify a syllabus algorithm, analyze its cost, implement it, test it, and adapt the choice when conditions change.

1 PROBLEM ABSTRACTION & MODELLING LO1 · model real world scenarios

Scenario / decision to solve: A software assurance team must decide when a syllabus algorithm / data structure becomes slow, memory-heavy or impractical. The lab generates test families, runs 13 algorithms, measures them and classifies risk as Low / Moderate / High.

Entities / inputs; size parameter(s): algorithm under test; generated keys, graphs, items, matrices, cities. Size: n, V, E, C, chain length

Objective + constraints: find the workload where each technique becomes impractical. Limits: 5 s time-out, size caps, seeded reproducible inputs

Representation: [x] graph [x] optimization [x] state/search [x] DP [x] data structure Other: benchmark + risk model

2 ALGORITHM CHOICE & FRAMEWORK DESIGN LO2 · choose & compare

Implemented DAA syllabus technique(s): Binomial and Fibonacci Heap, B-Tree, Red-Black Tree, Prim, Kruskal, Fractional and 0/1 Knapsack, Floyd-Warshall, Matrix Chain, N-Queens, Hamiltonian cycle (backtracking), TSP (Branch & Bound)

Alternative compared: Prim vs Kruskal; Fractional vs 0/1; Binomial vs Fibonacci; B-Tree vs RB-Tree **Selection reason / decision rule:** same seeded input for both; by density (E vs V), divisibility of items, operation mix (decrease-key heavy), disk vs memory

Workflow: scenario → model → choose → execute → measure → explain Architecture diagram: report p. ____

3 COMPLEXITY & CORRECTNESS LO3 · reason formally

Major technique / operation	Expected time T(n)	Space S(n)	Assumption / correctness argument
Binomial / Fibonacci heap; B-Tree; Red-Black tree	O(log n) per op; Fib insert, decrease-key O(1) amortized	O(n)	Invariants checked after each run: sorted extractions, black-height, equal leaf depth
Prim (heap); Kruskal (sort + union-find)	O(E log V); O(E log E)	O(V+E)	Both MST costs must match; V-1 edges, connected (cut / cycle property)
Fractional / 0/1 Knapsack	O(n log n); O(nC)	O(n); O(nC)	Greedy-choice (fractional); optimal substructure (DP); capacity check
Floyd-Warshall; Matrix Chain	$\Theta(V^3)$; $\Theta(n^3)$	$\Theta(V^2)$; $\Theta(n^2)$	Counted ops = V^3 and $(n^3-n)/6$; diagonal / triangle and cost recomputation checks
N-Queens; Hamiltonian; TSP (B&B)	O(n!) worst case	O(n)	No attacking queens; each vertex once; tour cost recomputed

Complexity status (as applicable): [x] polynomial [x] pseudo-polynomial [x] NP-Hard optimization [x] NP-Complete decision [ ] N/A

4 IMPLEMENTATION & VALIDATION LO4 · build & verify

Language / tools and implemented modules: JavaScript (single HTML + CSS file, Canvas charts, localStorage). Modules: 13 algorithms, input generators, verifiers, step visualiser, benchmark mode, risk classifier, CSV/JSON export

Correctness tests: [x] normal [x] boundary [x] large [x] difficult Tests passed: 29 / 29 Code proof: p. ____

5 EXPERIMENTS & ADAPTATION LO5 · measure & adapt

Input / condition	Size n	Algorithm / structure	Runtime / ops / memory	Result / observation
Baseline: sparse graph (E=2V)	V=200	Prim / Kruskal	Prim 2.3 ms, 6,585 ops, ~16 KB; Kruskal 0.9 ms, 3,678 ops, ~11 KB	Same MST cost (5327); Kruskal needs fewer ops
Changed: dense graph (E=15,920)	V=200	Prim / Kruskal	Prim 15.1 ms, 68,965 ops, ~513 KB; Kruskal 4.8 ms, 166,218 ops, ~384 KB	Same cost (265); Prim far fewer ops, Kruskal faster in JS (native sort)
Scaling: Floyd-Warshall	V=10 to 100	Floyd-Warshall	1,000 to 1,000,000 ops; 0.4 to 22 ms; ~1.6 to 160 KB	Ops x1000 = (x10)^3, as $\Theta(V^3)$ predicts
Growth: all-solutions N-Queens	n=8 to 11	Backtracking	States 15,720 to 1,806,706; 2.3 to 73 ms	About x115 states; risk MODERATE at n=11
Greedy failure (capacity 10)	3 items	Fractional vs 0/1	Fractional 19.2; 0/1 DP 18; greedy-by-ratio 0/1 = 12	Greedy is optimal only when items are divisible

Decision / behaviour changed because: density: on dense graphs Prim does about 2.4x fewer operations; Kruskal wins on sparse. 0/1 needs DP because greedy gives 12 vs optimum 18 Evidence: fig./table p. ____

6 EXPLANATION, LIMITS & DEMONSTRATION LO5 · interpret & communicate

Main finding and one limitation / scalability concern: Polynomial algorithms stay LOW risk at tested sizes and match their bounds; backtracking grows ~x100 states per +2 sizes (MODERATE at n=11). Limits: 0/1 DP memory O(nC), no tree deletion, no Web Worker, timings machine-dependent
Theory versus observation / reason for difference: Floyd-Warshall does exactly V^3 and Matrix Chain exactly $(n^3-n)/6$ ops; wall time grows slightly faster (cache, JIT). TSP B&B visited ~51k states at 12 cities vs 11! = 39.9M tours, so pruning beat the worst case
Explain NP-Hard / NP-Complete implications (if applicable): TSP optimization is NP-Hard (decision NP-Complete); Hamiltonian cycle and 0/1 Knapsack decision are NP-Complete; 0/1 DP is pseudo-polynomial; Fractional is in P; standard N-Queens is not labelled NP-Complete. Exact search is limited to modest n

SUBMISSION CHECK [x] integrated framework [x] working code [x] alternative compared [x] time/space analysis [x] 4 test families [x] adapted decision [ ] report + demo

ASSESSOR: Modelling __/5 | Selection __/5 | Analysis __/5 | Code __/5 | Experiments __/5 | Demo __/5 TOTAL __/30

Faculty feedback: _____